

Kubernetes Authentication

1. What is Authentication?

Authentication means checking and verifying WHO is making a request to the Kubernetes cluster.

Easy example:

When you enter a school, the guard asks:

"Who are you?"

You show your ID.

The guard verifies your identity.

That's **authentication**.

 **Easy memory:**

Authentication = WHO are you? 

2. Who Handles Authentication in Kubernetes?

The **Kubernetes API Server** handles authentication of requests.

Remember:

 **API Server = Front Door of Kubernetes**

When someone wants to access Kubernetes:

User / Client

|



API Server

|



Authentication

|



Is this request from a valid identity?

The API Server authenticates the request **before allowing it to proceed**.

3. How Do Users Access Kubernetes?

Your notes say:

All users are managed by the API Server, either through kubectl or directly.

For example:

User

|



kubectl

|



API Server

|



Authentication

kubectl is simply a command-line tool used to communicate with the Kubernetes API Server.

4. Important Point ★

API Server authenticates FIRST

When a request comes to Kubernetes:

Request

|



API Server

|



Authentication

|



Authorization

|



Request allowed/rejected

So don't think:

"The Pod authenticates the user."

✗ No.

The **API Server** is responsible for authenticating API requests.

5. Does Kubernetes Have a Normal Username & Password?

This is where your notes are important.

Kubernetes does **not use a built-in normal username/password database for human users** in the way a typical website does.

So you might wonder:

"If I didn't create a username and password, how does Kubernetes know who I am?"

Kubernetes can use different authentication mechanisms.

Your notes specifically mention:

Static Password File

6. Static Password File

A **static password file** is a file on the API Server containing authentication information.

Conceptually:

Static Password File

username	password	user-id
----------	----------	---------

ali	password1	1001
-----	-----------	------

ahmed	password2	1002
-------	-----------	------

The API Server can be configured to use this file to authenticate requests.

So the flow is:

User

|

| username + password

▼

API Server

|

▼

Static Password File

|

▼

Credentials match?

|

|— YES  → Authenticated

|

|— NO  → Rejected

7. Why Is It Called "Static"?

Because the information is stored in a **file** and is not dynamically managed through the Kubernetes API.

Think:

 **Static file = information stored in a file**

8. Authentication ≠ Authorization

This is VERY important for KCNA.

Suppose I provide valid credentials.

Authentication asks:

"Who are you?"

 User is identified.

Then Kubernetes asks:

Authorization asks:

"What are you allowed to do?"

For example:

Authentication:

"I know you are Ali." 

Authorization:

"Ali can view Pods." 

"Ali can delete Nodes." 

So:

Authentication → WHO?

Authorization → WHAT can you do?

9. Complete Example

Suppose you run:

```
kubectl get pods
```

The flow is roughly:

You

|

| kubectl get pods



API Server

|



 Authentication

|

| "Who is making this request?"



Identity verified 

|



 Authorization

|

| "Is this user allowed to get Pods?"



Permission checked

|



Pods returned

If authentication fails:

Authentication ❌

|



Request rejected

The request doesn't proceed as an authenticated request.

★ Your Notes — Simplified for Revision

Write these in your KCNA notes:

Kubernetes Authentication

- **Authentication = verifying who is making a request.**
- The **API Server** handles authentication of Kubernetes API requests.
- Users access Kubernetes through **kubectl** or directly through the API.
- The **API Server authenticates the request before it is processed further.**
- Kubernetes does not provide a normal built-in username/password user database for human users.
- One authentication method is a **static password file**.
- Static password authentication uses credentials stored in a file.
- After authentication, **authorization** determines what the user is allowed to do.

🧠 Remember:

Authentication = WHO are you? 👤

Authorization = WHAT can you do? 🔑

Most important flow:

Request



API Server



Authentication 



Authorization 



Access Kubernetes Resource

One small clarification: Kubernetes can authenticate **both human users and workloads/service accounts**, but it does not maintain a built-in database of ordinary human users with username/passwords. Your course note's point is that human identities are generally supplied through authentication mechanisms rather than being created as Kubernetes user objects.